

Multiclass Sentiment Classification with Kafka Streaming

Group Members: Subhadip Maity, Mohammad Shahbaz Alam
Institution: University of Naples Federico II
Professor: Giancarlo Sperli
Course: Hardware and Software for Big Data Mod A
Project Type: Multiclass Sentiment Classification with Kafka Streaming

1. Challenge Description

1.1 Problem Statement

The objective of this project is to implement a multiclass sentiment classification system capable of analyzing textual data in real-time. The system classifies tweets into four sentiment categories: positive, negative, litigious, and uncertainty. This task addresses a fundamental challenge in Natural Language Processing (NLP) and Big Data processing: handling large-scale streaming text data with low-latency predictions.

1.2 Dataset

The project utilizes the Sentiment Dataset with 1 Million Tweets sourced from Kaggle. The dataset comprises:

Attribute	Value
Total Records	5,000 training samples + streaming test samples
Features	Raw tweet text and associated sentiment labels
Classes	4 sentiment categories (positive, negative, litigious, uncertainty)
Format	CSV with columns: Text (tweet content) and Label (sentiment label)
Data Source	kaggle.com/datasets/tariqsays/sentiment-dataset-with-1-million-tweets

Table 1: Dataset Overview.

1.3 Project Constraints

The project addresses the following mandatory requirements:

- Requirement 1: Using Apache Spark with Python for distributed data processing.
- Requirement 2: Using Apache Kafka as the stream processor for real-time data ingestion.
- Requirement 3: Implementing multiclass classification using SparkML and Spark-NLP.
- Requirement 4: Evaluating model performance using accuracy, precision, and recall metrics.

1.4 System Architecture Overview

The system consists of three key components:

- **Producer Component:** Reads pre-labeled tweets from CSV and publishes them to Kafka topic.
- **Consumer Component:** Subscribes to Kafka stream, applies machine learning model, and logs results.
- **Model Component:** Trained sentiment classifier using Spark ML Pipeline with NLP features.

The proposed architecture follows a Lambda architecture pattern optimized for real-time stream processing, with a batch training phase followed by continuous streaming inference.

2. Description of Proposed Methodology and Architecture

2.1 System Architecture

2.1.1 Data Flow Pipeline

The sentiment classification system operates in three distinct phases:

- **Phase 1 — Training (Batch Processing):** The system loads 5,000 training samples from CSV into Apache Spark to build the ML model before streaming begins.
- **Phase 2 — Streaming Inference:** After training, the consumer listens to Kafka topic "sentiment-tweets" and processes incoming micro-batches with prediction and evaluation.
- **Phase 3 — Metrics Aggregation:** Performance metrics from each batch are aggregated to track cumulative accuracy over time.

2.2 Model Architecture

The sentiment classification model follows a pipeline-based approach combining Spark NLP and Spark MLlib:

Stage	Component	Description
1	DocumentAssembler	Converts raw text into Document type required by Spark NLP.
2	UniversalSentenceEncoder	Pre-trained deep learning model generating 512-dimensional embeddings.
3	EmbeddingsFinisher	Extracts embeddings into vector format for ML algorithms.
4	SQLTransformer	Selects first embedding vector as feature for classification.
5	StringIndexer	Encodes string labels into numerical indices (0, 1, 2, 3).
6	LogisticRegression	Multiclass classifier operating on embedding features.
7	IndexToString	Converts predicted indices back to readable sentiment labels.

Table 2: Spark ML Pipeline Components.

2.3 Feature Extraction: Universal Sentence Encoder

The Universal Sentence Encoder (USE) provides advanced feature extraction:

- Pre-trained on large text corpora (Google repository).
- Generates fixed-size 512-dimensional embeddings capturing semantic meaning.
- Semantically similar tweets produce embeddings with high cosine similarity.
- Outperforms traditional TF-IDF and word2vec approaches for short texts.
- Supports multiple languages and domain-specific terms.

2.4 Classification Algorithm

Logistic Regression is selected as the classifier due to its proven effectiveness on pre-computed dense embeddings, fast training and inference, interpretable decision boundaries, and natural extension to multiclass (multinomial logistic regression).

Hyperparameters: Maximum iterations: 10 | Regularization: L2 (default) | Strategy: Multinomial

2.5 Producer Implementation

The producer component (producer.py) implements:

- Establishes connection to Kafka broker at localhost:9092.
- Reads the CSV dataset from ../data/tweets.csv using a robust cross-platform path.
- Skips first 5,000 valid rows reserved for model training.
- Sends remaining tweets as JSON messages to topic "sentiment-tweets".
- Implements rate limiting (50ms between messages) for realistic streaming simulation.

Label Normalization:

Raw Label Variants	Normalized Label
'positive', 'pos', '1', '1.0'	positive
'negative', 'neg', '0', '0.0'	negative
'litigious'	litigious
'uncertainty', 'uncertain'	uncertainty

Table 3: Label Normalization Mapping.

2.6 Consumer Implementation

Training Phase:

- Load first 5,000 training samples from CSV.
- Filter to 4 valid classes: positive, negative, litigious, uncertainty.
- Build complete NLP + ML pipeline and fit on training data.
- Persist model for streaming inference.

Streaming Phase:

- Subscribe to Kafka topic using structured streaming.
- Parse JSON messages into Text and Label columns.
- Process micro-batches using foreachBatch function.
- Apply trained model and evaluate using SparkML MulticlassClassificationEvaluator.
- Log results to experiment_results.csv and track cumulative accuracy.

2.7 Key Technical Features

- **Stream Processing Resilience:** Checkpoint location stored in ./checkpoint for fault recovery. failOnDataLoss=false prevents crashes on Kafka offset loss.
- **Handling Unseen Labels:** StringIndexer uses handleInvalid='keep' to gracefully handle unseen classes during streaming.
- **Memory Management:** Micro-batches use .persist() with explicit .unpersist() after processing. Spark driver memory: 4GB.

Parameter	Value
Training Set Size	5,000 samples
Total Batches Processed	438 micro-batches
Batch Processing Mode	Append (accumulative evaluation)
Kafka Partition	1 (single topic)
Spark Version	3.5.0 with Kafka connector
Spark-NLP Version	5.5.3
Spark Driver Memory	4GB

Table 4: Experimental Configuration.

3. Experimental Results

3.1 Experimental Setup

The sentiment classification model was evaluated on 438 consecutive streaming batches. Batch 1 is excluded from statistical analysis as it represents pipeline initialization (all metrics = 0.0), a known artifact of Spark Structured Streaming startup. The remaining 437 batches form the basis of all reported metrics.

3.2 Overall Performance Summary

Metric	Value (excl. Batch 1)	Range
Mean Accuracy	69.00%	40.00% - 100.00%
Mean Precision	72.79%	40.00% - 100.00%
Mean Recall	69.00%	40.00% - 100.00%
Final Cumulative Accuracy	69.00%	-
Peak Cumulative Accuracy	75.32%	Batch 6

Table 5: Performance Metrics Summary.

3.3 Detailed Results Analysis

3.3.1 Accuracy Trends

- **Initial Batches (2-20):** Mean accuracy 71.99%, demonstrating early stability after model warmup.
- **Mid-range Batches (100-200):** Mean accuracy 68.48%, showing consistent performance.
- **Final Batches (300-438):** Mean accuracy 67.72%.
- **Peak Performance:** 100% accuracy achieved in 1 batch.
- **Minimum Performance (excl. Batch 1):** 40.00% - reflects small batches with challenging label distributions.

3.3.2 Precision-Recall Trade-off

The precision and recall metrics demonstrate balanced classification performance. Mean Precision (72.79%) is slightly higher than Mean Recall (69.00%), indicating a low false-positive rate across all four sentiment classes.

3.3.3 Cumulative Accuracy Evolution

- **Batch 1 (excluded):** 0.00% - pipeline initialization artifact.

- Peak Cumulative Accuracy: 75.32% at Batch 6.
- Stabilization Region (Batch 50+): Plateaus around 69-70%.
- Final Cumulative Accuracy (Batch 438): 69.00%.
- Convergence Pattern: Shows decreasing variance after Batch 100.

3.3.4 Batch Size Variability

- Standard Deviation: 10.11%.
- Coefficient of Variation: 14.65% - indicates moderate variance across batches.
- High-Performance Batches: 149 batches with accuracy above 75%.

3.4 Performance Metrics by Class

Metric	SparkML Function	Mean Value
Accuracy	MulticlassClassificationEvaluator (metricName="accuracy")	69.00%
Precision	MulticlassClassificationEvaluator (metricName="weightedPrecision")	72.79%
Recall	MulticlassClassificationEvaluator (metricName="weightedRecall")	69.00%

Table 6: Evaluation Metrics Summary.

3.5 Key Performance Observations

3.5.1 Model Strengths

- Consistent Processing: 438 consecutive batches processed without errors.
- Precision Focus: Mean precision (72.79%) higher than recall, indicating low false-positive rate.
- Robust Streaming: Kafka-Spark integration handled streaming data reliably throughout.
- Semantic Embeddings: Universal Sentence Encoder provides rich 512-dimensional representations.

3.5.2 Challenges and Variance

- Batch Variance: Accuracy fluctuation (40%-100%) reflects variable batch sizes and label distributions.
- 4-Class Complexity: Distinguishing litigious and uncertainty from positive/negative is inherently harder than binary classification.
- Batch 1 Artifact: First batch shows 0.0 metrics due to Spark Structured Streaming pipeline initialization; excluded from analysis.
- Streaming Latency: Real-time predictions may encounter tweets with ambiguous sentiment.

3.6 Comparative Metrics

- Average processing speed: 100-500 tweets per micro-batch.
- Latency: <1 second per batch on standard hardware.
- No performance degradation observed over 438 batches.

3.7 Results Conclusion

The experimental results validate the successful implementation of the sentiment analysis system respecting all project constraints:

- **Constraint 1 - Spark with Python:** Successfully using PySpark API with Logistic Regression from MLlib.
- **Constraint 2 - Kafka Stream Processor:** Successfully processing streaming data from Kafka topic "sentiment-tweets".
- **Constraint 3 - Multiclass Classification:** Achieved with 4 sentiment classes (positive, negative, litigious, uncertainty) using multinomial logistic regression.
- **Constraint 4 - Evaluation Metrics:** All three required metrics (accuracy, precision, recall) computed using SparkML evaluators.

The 69.00% final cumulative accuracy demonstrates solid baseline performance for 4-class sentiment classification from streaming data. Future improvements could include ensemble methods, hyperparameter tuning, and addressing class imbalance between the four categories.

References

- [1] Kaggle. (2024). Sentiment Dataset with 1 Million Tweets. <https://www.kaggle.com/datasets/tariqsays/sentiment-dataset-with-1-million-tweets>
- [2] Marz, N., & Warren, J. (2015). Big Data: Principles and best practices of scalable real-time data systems. Manning Publications.
- [3] Cer, D., Yang, Y., Kong, S. Y., et al. (2018). Universal Sentence Encoders. arXiv:1803.11175.
- [4] Apache Spark. (2024). MLlib: Machine Learning Library. <https://spark.apache.org/mllib/>
- [5] Apache Spark. (2024). Evaluation Metrics - MLlib. <https://spark.apache.org/docs/latest/mllib-evaluation-metrics.html>
- [6] Apache Spark. (2024). PySpark API Documentation. <https://spark.apache.org/docs/latest/api/python/>
- [7] Apache Kafka. (2024). Kafka Python Client. <https://github.com/dpkp/kafka-python>